DESARROLLO DE APLICACIÓN MÓVIL “MECANICSYNC” PARA LA GESTIÓN DE TALLERES MECÁNICOS

PROYECTO FIN DE GRADO DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA. (I.E.S. JUAN BOSCO, ALCÁZAR DE SAN JUAN).

Sandra Fernández Sorribes

CONTENIDO

Introducción. 2
1.1. Presentación del proyecto. 2
1.2. Objetivos del proyecto. 2
1.3. Justificación del proyecto. 2
2. Análisis de requerimientos. 2
2.1. Identificación de necesidades y requerimientos. 2
2.2. Identificación del público objetivo. 3
2.3. Estudio de mercado y competencia. 3
3. Diseño y planificación. 4
3.1. Desarrollo de las funcionalidades del proyecto. 4
3.2. Pruebas unitarias y de integración. 7
3.3. Corrección de errores y optimización. 15
4. Documentación. 15
4.1. Documentación técnica. 16
4.2. Documentación de usuario final. 19
4.3. Manual de instalación y configuración. 27
4.4. Consideraciones adicionales. 29
5. Mantenimiento y evolución. 29
5.1. Identificación de posibles mejoras y evolución del proyecto. 29
5.2. Mejoras futuras. 30
6. Conclusiones. 30
6.1. Evaluación del proyecto. 30
6.2. Cumplimiento de objetivos y requisitos. 30
6.3. Lecciones aprendidas y recomendaciones para futuros proyectos. 30
7. Bibliografía/Webgrafía y referencias. 32
7.1. Fuentes utilizadas en el proyecto. 32
7.2. Herramientas utilizadas. 32

INTRODUCCIÓN.

1.1. Presentación del proyecto.

MecanicSync es una aplicación Android conectada con una base de datos MySQL que permite al personal de los talleres mecánicos gestionar de forma eficiente a sus clientes y el estado de sus reparaciones.

El sistema está pensado para facilitar y agilizar el control diario de un taller, ya que, desde un móvil o Tablet, los propios mecánicos serán capaces de registrar y actualizar el estado de las reparaciones que realizan. Esto mejora la organización del trabajo, ya que se puede conocer, en tiempo real, la carga de trabajo de los mecánicos, las horas estimadas de ocupación y si existe alguna incidencia en los trabajos. También ayuda a mejorar la comunicación entre el taller y sus clientes, ya que desde administración pueden dar información en tiempo real de las reparaciones.

1.2. Objetivos del proyecto.

- Digitalizar las reparaciones de un taller mecánico.
- Permitir un registro completo de clientes y vehículos.
- Llevar control del estado de cada reparación (pendiente, en progreso, finalizada, entregado).
- Implementar *login* seguro (*admin*/ mecánico).
- Incluir un área "Sobre mí" para mostrar mis datos profesionales.

1.3. Justificación del proyecto.

Muchas pequeñas empresas gestionan su día a día con libretas u hojas de Excel, además de centralizar estas tareas en una única persona, lo que genera errores y pérdidas de tiempo. Está aplicación propone una aplicación centralizada que sea fácil de utilizar y accesible, desde un dispositivo Android, para los empleados de un pequeño taller, de forma que propietario y empleados puedan llevar un control digital del negocio.

También se pretende la integración de toda la información en una misma base de datos para compartir y sincronizar las tareas entre los operarios del mismo, mejorando, tanto la coordinación, como el acceso a la información que será más rápido.

MecanicSync pretende ser una aplicación ágil, sencilla y con una interfaz intuitiva, de fácil manejo, orientada exclusivamente a las necesidades más básicas que pueda tener un taller mecánico en su día a día.

2. ANÁLISIS DE REQUERIMIENTOS.

2.1. Identificación de necesidades y requerimientos.

- Requerimientos funcionales.
  - Autentificación mediante *login* y *logout*.
  - CRUD completo de usuarios, con roles.

- CRUD completo de clientes.
- Registro de vehículos asociados a clientes.
- Gestión de las reparaciones asociadas a los clientes y vehículos y de su estado.
- Generación de reportes o listados (por fechas, clientes o estados).
- Pantalla "Sobre mí" con CV o enlace profesional.

- Requerimientos no funcionales.
  - Interfaz intuitiva y accesible.
  - Logotipo original.
  - Conexión segura a una base de datos (vía *API REST*).
  - Compatible con dispositivos Android.
  - Fácil mantenimiento y actualización.
  - Aplicación escalable.
  - Registro de fechas.
  - Documentación técnica y manual de usuario

2.2. Identificación del público objetivo.

Esta aplicación está diseñada para talleres mecánicos de pequeño o mediano tamaño que deseen mejorar y agilizar la gestión diaria de la empresa, sin necesidad de soluciones más complejas y costosas.

En esta fase del desarrollo del proyecto solo esta dirigida a los empleados de un taller, que no necesiten de altos conocimientos técnicos para su utilización. El único requisito de los usuarios será el de tener un dispositivo Android para la instalación de la aplicación.

El usuario "administrador" necesitará algún conocimiento técnico más, para dar de alta roles, pero aún así, la aplicación va dirigida a usuarios sin conocimientos informáticos.

2.3. Estudio de mercado y competencia.

El desarrollo de *MecanicSync* parte de la necesidad real existente en el sector de los talleres mecánicos de pequeña y mediana escala, donde en muchos casos la gestión de clientes, vehículos y reparaciones se realiza todavía mediante sistemas poco digitalizados, como hojas de cálculo, software genérico o incluso registros manuales.

Este contexto genera problemas habituales como la pérdida de información, falta de trazabilidad en las reparaciones, dificultad para conocer el estado del taller en tiempo real y escasa optimización de recursos.

En el mercado existen diferentes soluciones orientadas a la gestión de talleres mecánicos, principalmente en formato de software de escritorio o plataformas SaaS. Sin embargo, muchas de estas soluciones presentan limitaciones importantes para el tipo de usuario objetivo de este proyecto:

- Alta complejidad de uso
- Costes de licencia elevados
- Escasa adaptación a dispositivos móviles

- Interfaces poco intuitivas o poco actualizadas
- Dependencia de entornos de escritorio

A continuación, enumeramos algunas de las soluciones comerciales orientadas a la gestión integral de talleres mecánicos más representativas:

- PTME (Programa de Gestión de Taller Mecánico): Es uno de los softwares más utilizados en España para la gestión de talleres. Permite la administración de clientes, vehículos, órdenes de trabajo y facturación. Destaca por su enfoque en la rapidez operativa, permitiendo crear facturas y gestionar reparaciones en pocos minutos. Está orientado a talleres que buscan digitalizar procesos básicos sin una gran complejidad técnica.[1]
- TallerGP: Se trata de un ERP en la nube que permite gestionar de forma completa un taller mecánico desde cualquier dispositivo. Incluye módulos de facturación, órdenes de reparación, clientes y control administrativo. Su principal ventaja es que no requiere instalación y funciona directamente desde el navegador, lo que facilita su uso en diferentes sistemas operativos.[2]
- AutoSoft Taller: Software orientado a talleres de automoción con funcionalidades avanzadas como gestión de órdenes de trabajo, presupuestos, historial de vehículos y control de beneficios. Destaca por incorporar automatización de procesos y funcionalidades avanzadas como asistencia mediante inteligencia artificial.[3]

Estos sistemas demuestran que existe una tendencia clara hacia la digitalización de los talleres mecánicos mediante soluciones SaaS o ERP. Sin embargo, muchas de estas herramientas están orientadas a modelos de pago, interfaces complejas o entornos de escritorio, lo que abre la oportunidad a soluciones más modernas, móviles y adaptadas al trabajo diario en taller como *MecanicSync*.

3. DISEÑO Y PLANIFICACIÓN.

En este apartado se describe el diseño conceptual del sistema *MecanicSync*, definiendo la estructura general, las funcionalidades principales y la arquitectura empleada. La implementación técnica detallada se desarrolla en el apartado 4.

3.1. Desarrollo de las funcionalidades del proyecto.

El sistema *MecanicSync* se ha diseñado con el objetivo de cubrir las necesidades principales de gestión de un taller mecánico, estructurándose en distintos módulos funcionales.

- Gestión de usuarios y control de acceso

Se ha definido un sistema de autenticación basado en usuario y contraseña que permite el acceso seguro a la aplicación.

[1] https://www.programatallermecani.com/
[2] https://gdtaller.com/
[3] https://autosofttaller.com/index.html

Se contemplan dos tipos de roles:

- Administrador, con acceso completo a todos los módulos.
- Mecánico, con acceso limitado a la gestión de reparaciones e histórico.

Esta diferenciación permite controlar los permisos de acceso a las distintas funcionalidades de la aplicación. Más adelante, en la documentación para el usuario final, incidiremos más en las diferencias que se derivan de cada acceso.

- Gestión de clientes

El sistema permite registrar, consultar, modificar y eliminar clientes del taller.

Cada cliente puede tener asociados uno o varios vehículos, lo que facilita el seguimiento de su actividad dentro del sistema.

- Gestión de vehículos

Se ha diseñado un módulo para la gestión de vehículos asociados a los clientes, incluyendo información como matrícula, marca y modelo.

Esto permite mantener un control detallado de los vehículos y su relación con las reparaciones.

- Gestión de reparaciones

El sistema permite registrar y gestionar las reparaciones realizadas sobre los vehículos.

Se definen los siguientes estados:

- Pendiente
- En proceso
- Finalizada
- Entregada

Este módulo permite conocer en todo momento el estado del trabajo en el taller.

- Visualización del estado del taller

Se ha diseñado una pantalla principal (*dashboard*) que permite visualizar en tiempo real:

- Número de clientes con vehículos en el taller
- Número de vehículos en proceso
- Estado de las reparaciones
- Esto proporciona una visión global del funcionamiento del taller.

- Módulo "Sobre mí"

Se incluye un apartado adicional que permite mostrar información profesional del desarrollador mediante el acceso a su currículum vitae.

- Diseño de la Base de Datos. MecanicSync

La base de datos de *MecanicSync* ha sido diseñada siguiendo un modelo relacional, con el objetivo de garantizar integridad de datos, flexibilidad y escalabilidad para gestionar talleres mecánicos de pequeño y mediano tamaño.

La estructura se centra en los actores principales del sistema: usuarios, clientes, vehículos y reparaciones, así como en los datos complementarios que permiten un control detallado del negocio.

El sistema se organiza en torno a entidades principales relacionadas entre sí, permitiendo modelar de forma coherente la lógica del negocio.

- Integridad referencial y normalización

La base de datos está normalizada hasta tercera forma normal (3FN), por lo que no existen datos redundantes y cada atributo depende únicamente de la clave primaria de su tabla.

Las relaciones entre entidades permiten garantizar la consistencia de los datos dentro del sistema.

- Justificación del diseño

La separación de roles y usuarios permite la escalabilidad y control de permisos. La aplicación está pensada para que en un futuro se añadan más tipos de roles como administración o clientes. Esto puede ayudar a las Pymes a llevar un control del envío de facturas o el estado de las mismas, y a los clientes a consultar las reparaciones hechas sobre su vehículo o las facturas emitidas a su nombre.

También sería interesante a futuro implementar un módulo para coger cita en el mecánico. Si cada mecánico tiene asociadas sus reparaciones y en la tabla tipo reparación ya aparecen las horas de trabajo estimado, se podría relacionar con un calendario para saber cuándo determinado mecánico puede estar libre para más reparaciones. También sería un buen método para el control horario de los trabajadores.

La normalización de marcas y modelos evita la duplicidad y facilita la actualización de la información.

El registro detallado de las reparaciones permite conocer el estado de cada trabajo en tiempo real y calcular los costes.

La consistencia y la fiabilidad de los datos está garantizada por la integridad referencial, clave en cualquier aplicación empresarial real.

- Diagrama ER

El modelo entidad-relación ha sido generado mediante ingeniería inversa utilizando MySQL Workbench, permitiendo visualizar de forma clara las relaciones entre las entidades del sistema.

El diagrama refleja todas las entidades y relaciones, mostrando gráficamente cómo los usuarios, clientes, vehículos y reparaciones interactúan.

Este diagrama facilita la comprensión del modelo y sirve como referencia técnica para desarrolladores y usuarios avanzados.

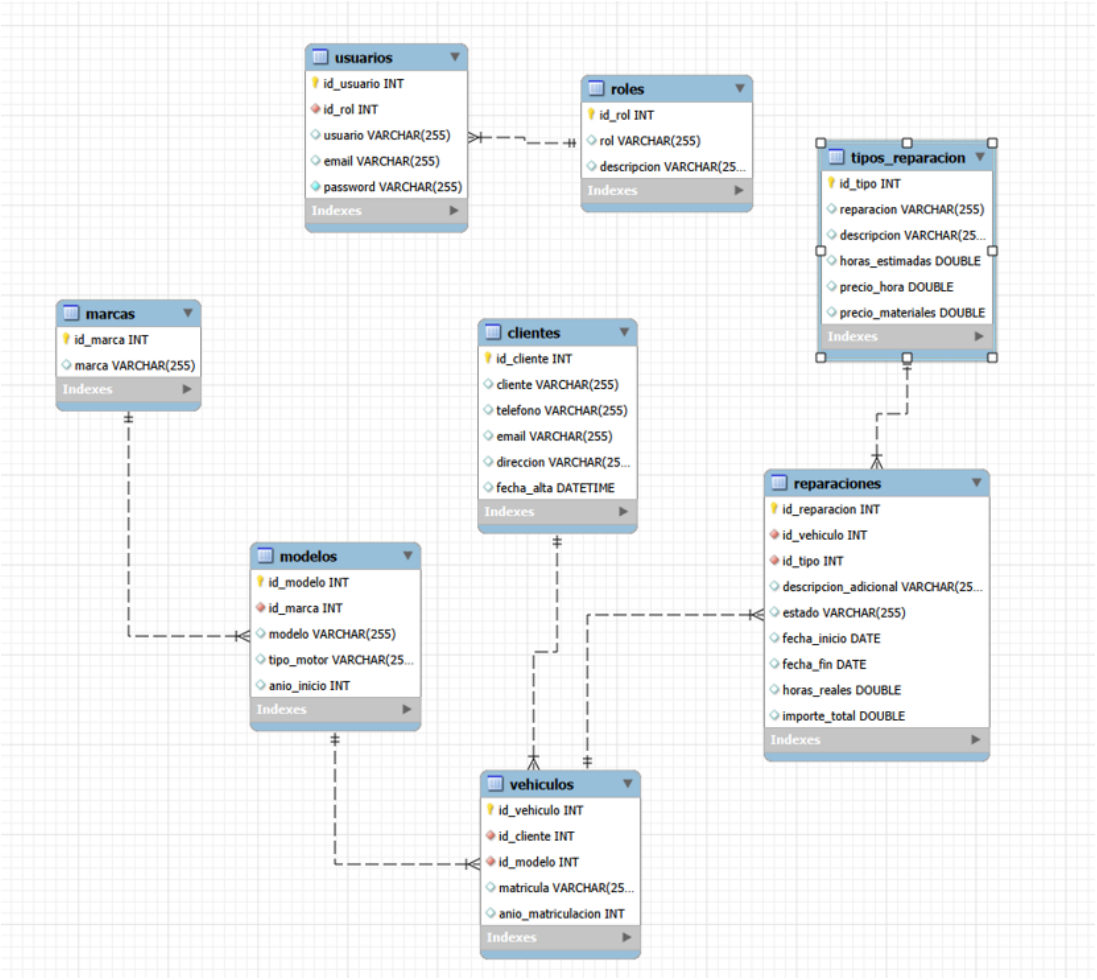


Figura 1: Diagrama entidad-relación

- Arquitectura del sistema:

Se ha optado por una arquitectura cliente-servidor compuesta por tres capas:

- Aplicación Android (cliente)
- *API REST* desarrollada con *Spring Boot* (servidor)
- Base de datos *MySQL* (persistencia)

Esta arquitectura permite desacoplar los componentes del sistema, facilitando su mantenimiento, escalabilidad y evolución.

3.2. Pruebas unitarias y de integración.

Con el objetivo de garantizar el correcto funcionamiento del sistema, se han realizado pruebas de integración sobre la *API REST* mediante la herramienta *Postman*. Estas pruebas han

permitido validar la correcta comunicación entre la aplicación Android, la API desarrollada en *Spring Boot* y la base de datos en *MySQL*.

Se han realizado pruebas sobre todos los *endpoints* implementados, cubriendo operaciones CRUD completas, así como validaciones de errores y casos límite.

- Pruebas sobre el módulo de vehículos.

Se han realizado pruebas completas sobre los *endpoints* del módulo de vehículos, verificando:

- Obtención de todos los vehículos
- Consulta por ID y matrícula
- Filtrado por cliente
- Creación, actualización y eliminación

Ejemplo de prueba: `GET /api/vehiculos`

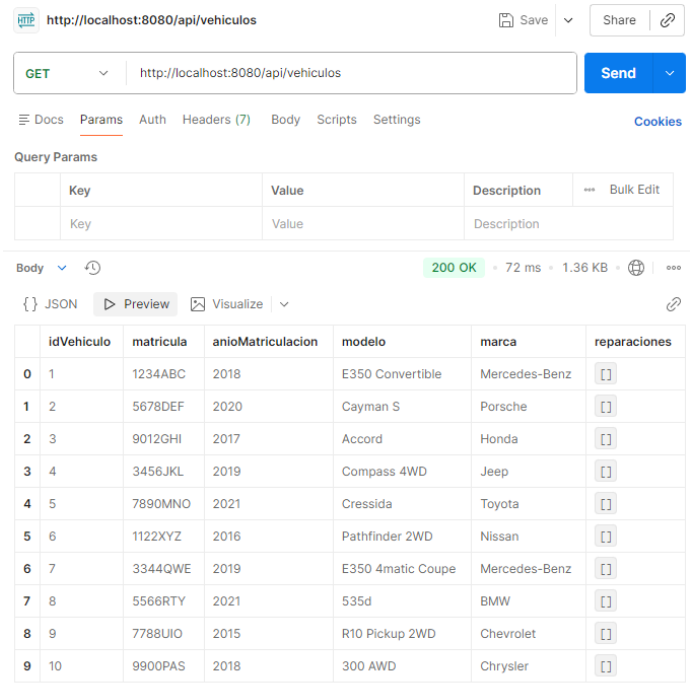


Figura 2: Prueba de consulta de todos los vehículos usando Postman

Resultado:

- Lista completa de vehículos
- Inclusión correcta de marca y modelo
- Formato *JSON* válido

También se han validado operaciones de escritura: `POST /api/vehículos`

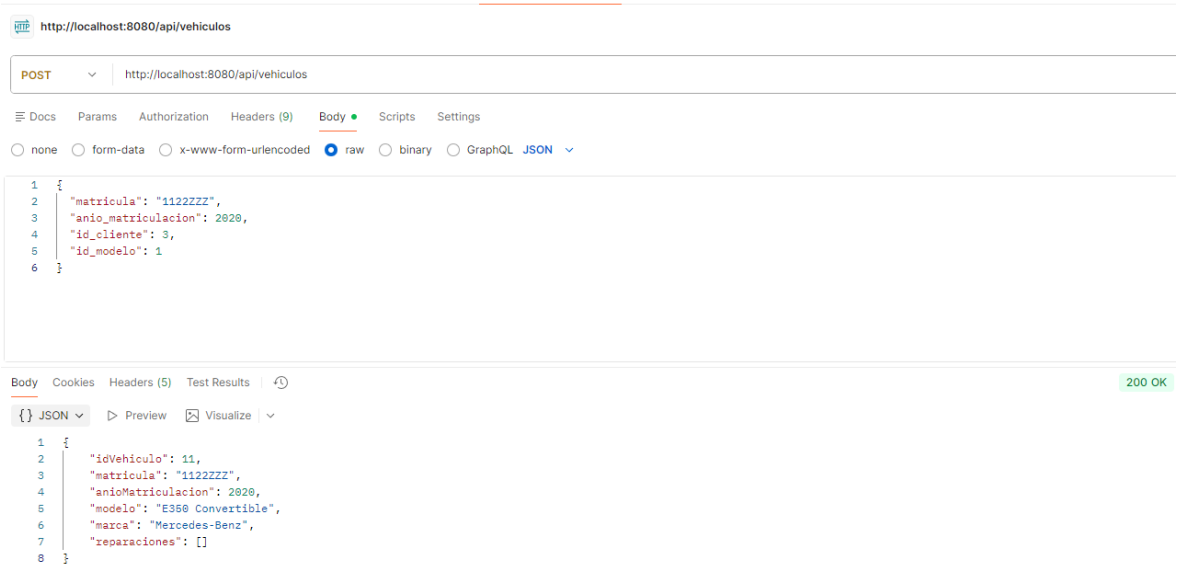
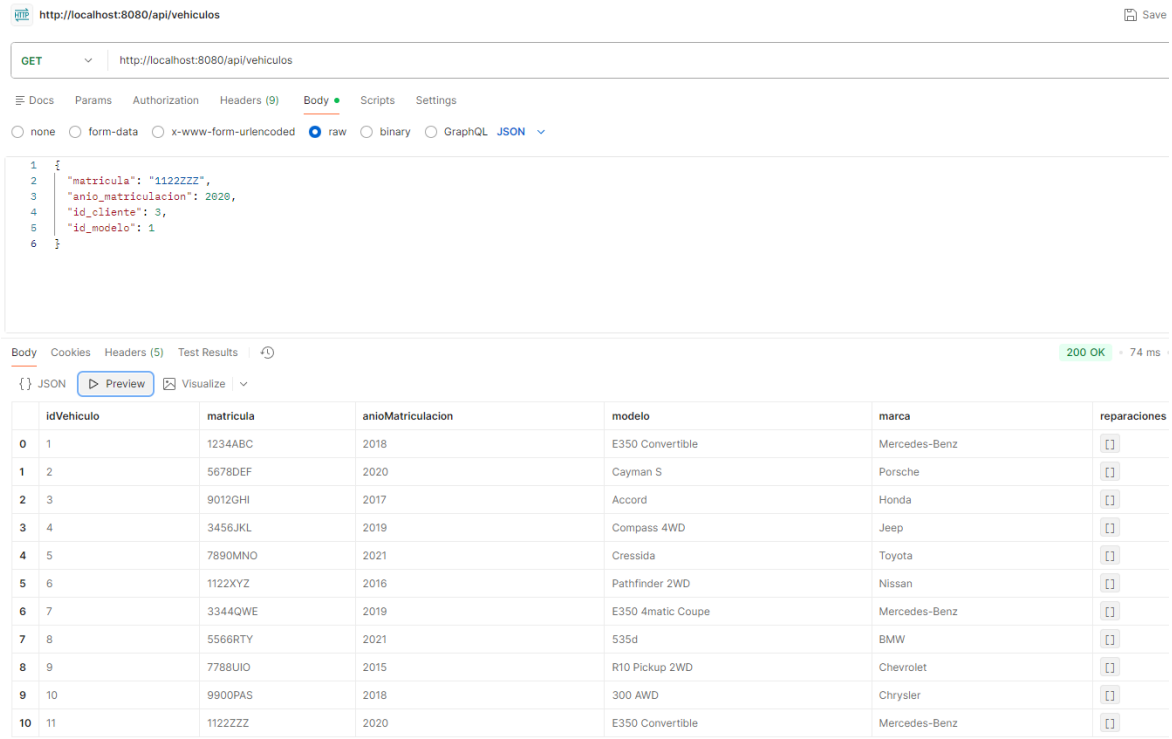


Figura 3: Prueba de inserción de nuevo vehículo en la base de datos usando *Postman*



	idVehiculo	matricula	anioMatriculacion	modelo	marca	reparaciones
0	1	1234ABC	2018	E350 Convertible	Mercedes-Benz	[]
1	2	5678DEF	2020	Cayman S	Porsche	[]
2	3	9012GHI	2017	Accord	Honda	[]
3	4	3456JKL	2019	Compass 4WD	Jeep	[]
4	5	7890MNO	2021	Cressida	Toyota	[]
5	6	1122XYZ	2016	Pathfinder 2WD	Nissan	[]
6	7	3344QWE	2019	E350 4matic Coupe	Mercedes-Benz	[]
7	8	5566RTY	2021	535d	BMW	[]
8	9	7788UIO	2015	R10 Pickup 2WD	Chevrolet	[]
9	10	9900PAS	2018	300 AWD	Chrysler	[]
10	11	1122ZZZ	2020	E350 Convertible	Mercedes-Benz	[]

Figura 4: Resultados de la base de datos posteriores a la inserción de un nuevo vehículo usando *Postman*.

Resultados:

- Creación correcta del recurso
- Generación automática del identificador
- Persistencia en base de datos

- Pruebas sobre el módulo de reparaciones

Se han realizado pruebas específicas para validar la lógica de negocio de las reparaciones:

- Consulta global y por ID
- Filtrado por vehículo y estado
- Creación y actualización de reparaciones
- Eliminación de registros

Ejemplo de prueba: `GET /api/reparaciones/estado/pendiente`

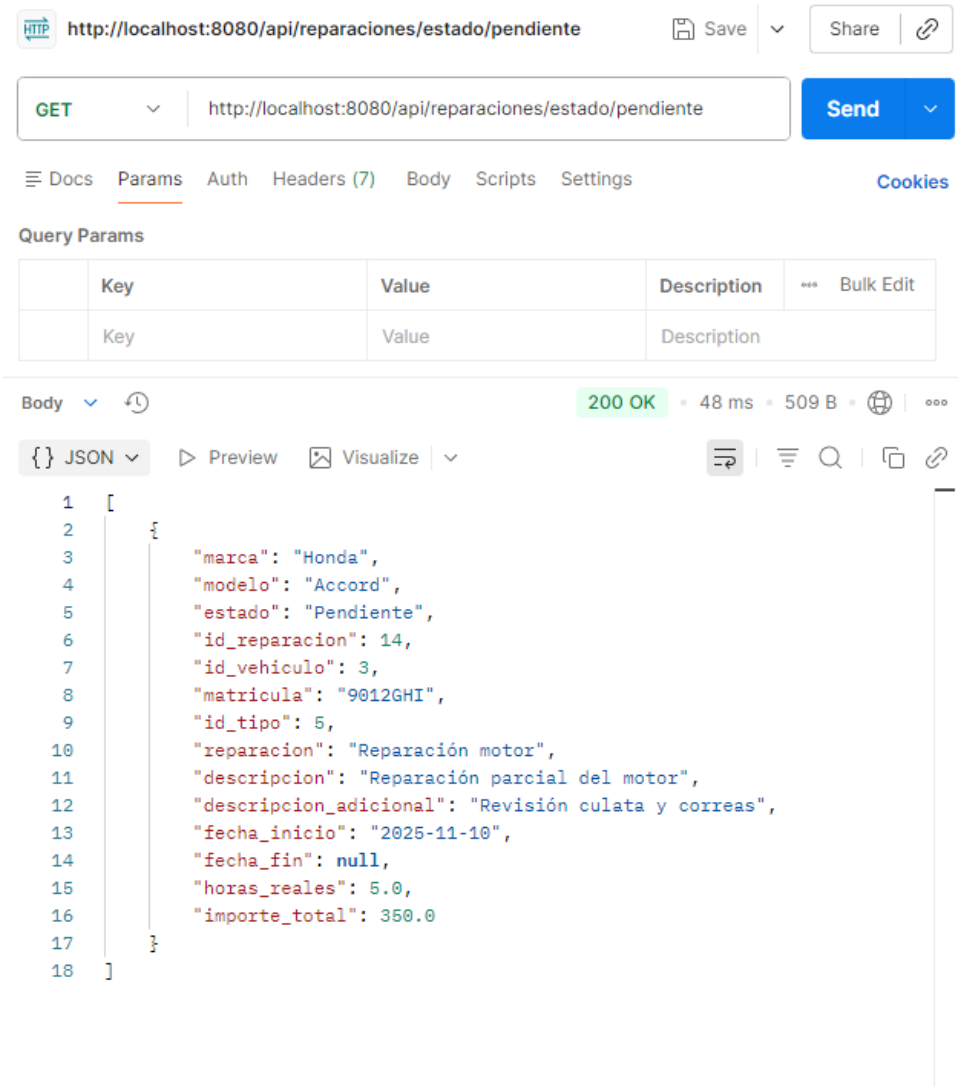


Figura 5: Prueba de consulta por estado de una reparación usando *Postman*

Resultado:

- Lista filtrada correctamente según estado

- Pruebas negativas y de control de errores.

Se han realizado pruebas negativas con el objetivo de validar el comportamiento del sistema ante situaciones de error.

Entre estas pruebas se incluye la creación de reparaciones con tipos inexistentes, donde la API devuelve correctamente un código 400 (*Bad Request*), evitando inconsistencias en la base de datos.

Asimismo, se han probado operaciones sobre recursos inexistentes, como la actualización y eliminación de reparaciones no registradas, obteniendo como respuesta un código 404 (*Not Found).*

Ejemplos:

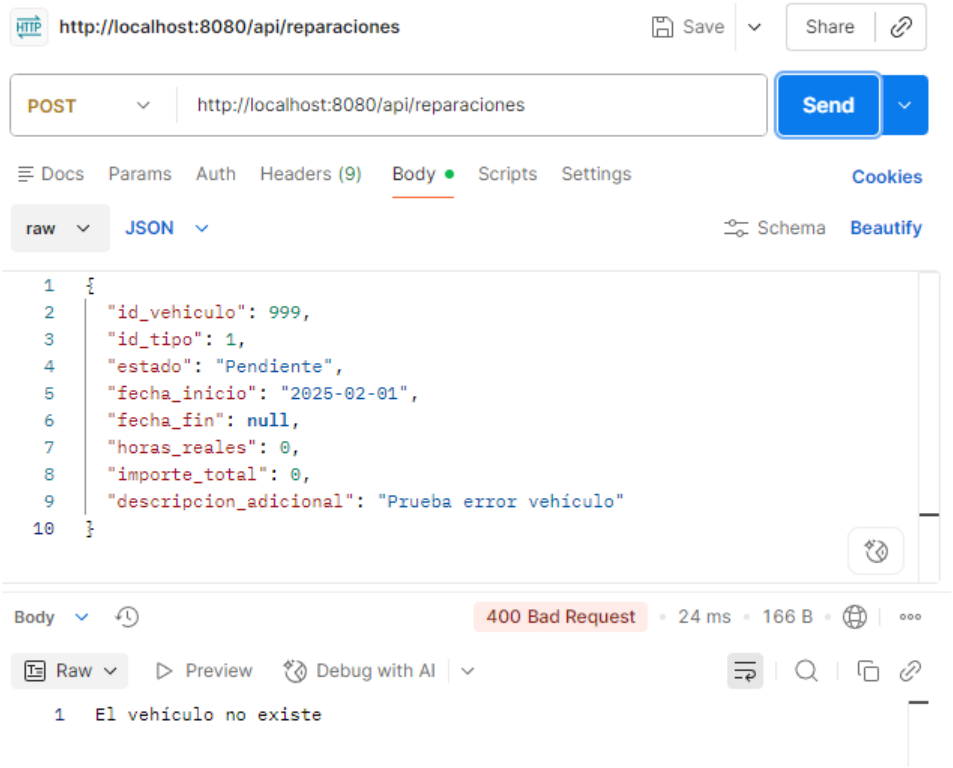


Figura 6: Prueba de creación de una reparación sobre un vehículo inexistente. Respuesta 400

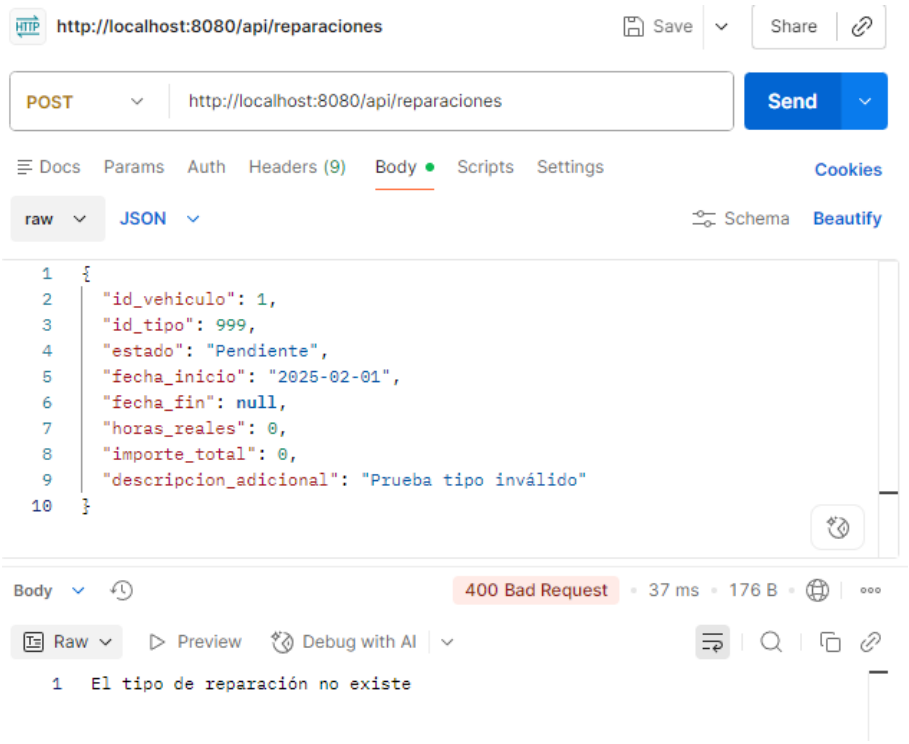


Figura 7: Prueba de creación de una reparación con tipo de reparación no válido. Respuesta 400

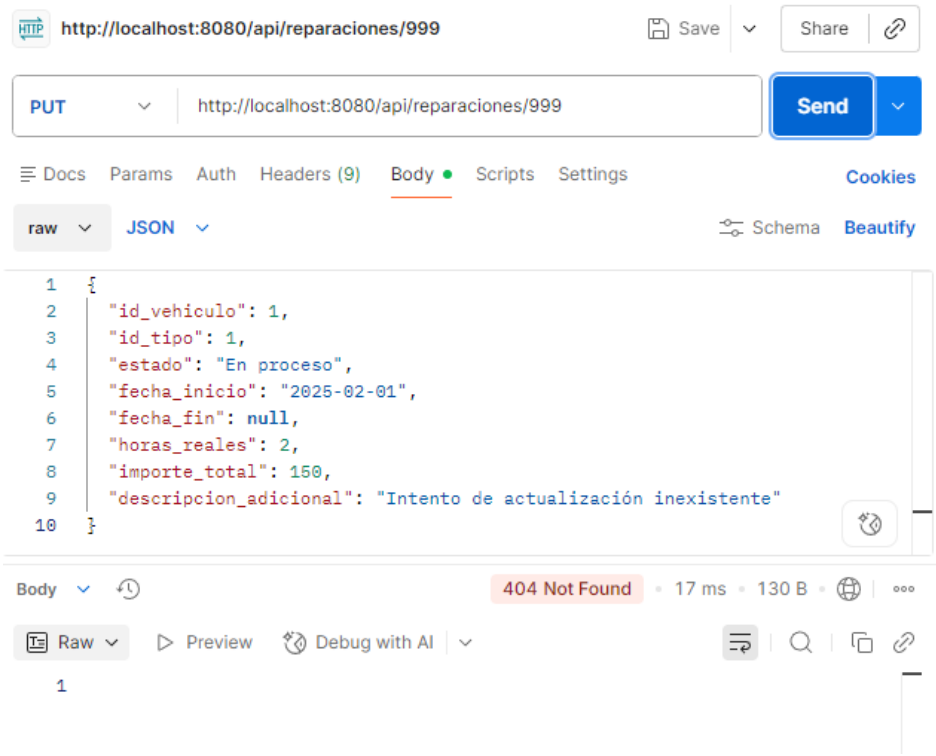


Figura 8: Prueba de actualización de un recurso inexistente. Respuesta 404

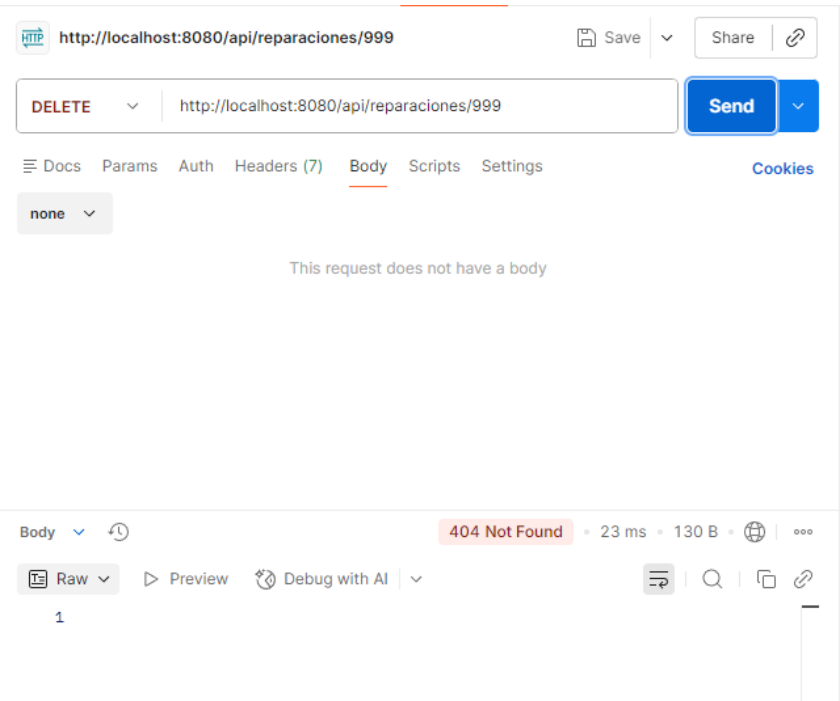


Figura 9: Prueba de eliminación de un recurso inexistente. Respuesta 404.

Estas pruebas confirman que la API implementa un control de errores adecuado y cumple con los estándares HTTP, mejorando la robustez y fiabilidad del sistema.

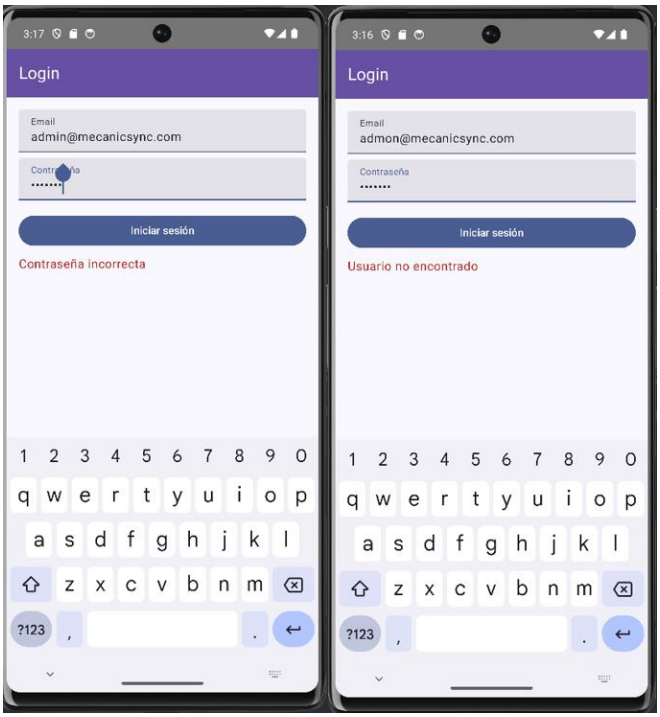
- Pruebas de interfaz de usuario (UI)

Además de las pruebas realizadas sobre la API, se han llevado a cabo pruebas manuales sobre la interfaz de usuario de la aplicación Android, con el objetivo de validar la correcta interacción del usuario con el sistema y la integración con los datos obtenidos desde la API. Muchas de las pruebas que se describen a continuación serán validadas en la presentación del proyecto, comprobando *insitu* la usabilidad de la aplicación.

Estas pruebas han permitido verificar:

- Proceso de autenticación (*login*):

Validación del acceso mediante usuario y contraseña, comprobando el comportamiento ante credenciales correctas e incorrectas.



- Navegación entre pantallas:

Comprobación del correcto funcionamiento del sistema de navegación, asegurando el acceso a los distintos módulos (usuarios, clientes, vehículos, reparaciones e histórico).

- Gestión de formularios:

Verificación de la creación, edición y eliminación de entidades (clientes, vehículos y reparaciones), asegurando que los datos introducidos se envían correctamente a la API y se reflejan en la interfaz.

- Visualización de datos:

Comprobación de que la información mostrada en pantalla (listas, detalles y *dashboard*) coincide con los datos almacenados en la base de datos.

- Control de roles:

Validación de la restricción de funcionalidades según el tipo de usuario (Administrador o Mecánico), garantizando que cada perfil accede únicamente a los módulos permitidos.

- Flujo completo de uso (*end-to-end*):

Simulación del uso real de la aplicación, desde el inicio de sesión hasta la gestión completa de clientes, vehículos y reparaciones.

Estas pruebas han permitido detectar y corregir posibles errores de usabilidad, navegación y visualización, mejorando la experiencia de usuario.

- Conclusión de las pruebas

Las pruebas realizadas permiten confirmar que:

- La API responde correctamente a todas las operaciones CRUD
- La integridad de los datos se mantiene en todo momento
- El sistema gestiona adecuadamente los errores
- La comunicación cliente-servidor funciona de forma estable
- La interfaz de usuario responde correctamente a las acciones del usuario.

Esto garantiza la fiabilidad del sistema en un entorno real de uso.

3.3. Corrección de errores y optimización.

Durante el desarrollo del proyecto se han identificado y corregido diversos errores tanto en la API REST como en la aplicación Android, mejorando la estabilidad y el rendimiento del sistema.

- Corrección de errores

Se han abordado diferentes tipos de errores:

- Errores de comunicación cliente-servidor: Problemas en la serialización y deserialización de datos JSON, corregidos mediante el uso de DTOs y mapeos adecuados.
- Errores de integridad de datos: Validaciones para evitar referencias a entidades inexistentes (vehículos o tipos de reparación).
- Errores de navegación en la aplicación Android: Ajustes en el sistema de navegación para evitar pantallas en blanco o rutas incorrectas.
- Errores en la gestión de estados: Corrección de inconsistencias en el estado de las reparaciones (pendiente, en proceso, finalizada).
- Errores en la visualización de datos: Ajustes en el mapeo de datos para garantizar que la información mostrada en la UI es correcta.

- Optimización del rendimiento

Se han aplicado diversas mejoras orientadas a optimizar el rendimiento del sistema:

- Uso de DTOs en la API: Reduce el tamaño de las respuestas y evita sobrecarga innecesaria de datos.
- Consultas optimizadas en base de datos: Uso de relaciones bien definidas y consultas específicas para evitar cargas innecesarias.
- Gestión eficiente de llamadas a la API: Implementación de repositorios en Android para centralizar el acceso a datos.
- Uso de arquitectura MVVM: Permite separar la lógica de negocio de la interfaz, mejorando el rendimiento y mantenimiento.
- Carga dinámica de datos en el Dashboard: Solo se solicitan los datos necesarios en cada momento.

4. DOCUMENTACIÓN.

4.1. Documentación técnica.

- Base de datos:

La base de datos de *MecanicSync* "basefernandez" se ha diseñado para gestionar de forma eficiente toda la información relacionada con un taller mecánico: usuarios, clientes, vehículos, reparaciones y los distintos tipos de servicio.

Se ha creado una tabla "roles" que define los diferentes tipos de usuarios del sistema. Actualmente los dos tipos de roles existentes son los de Administrador y mecánico. Permite controlar los niveles de acceso y permisos dentro de la aplicación.

La tabla "usuario" almacena información de cada usuario, incluyendo nombre, correo electrónico, contraseña, rol asignado y la fecha de creación de la cuenta.

Cada usuario se asocia a un rol mediante una clave foránea (id_rol), permitiendo de este modo implementar la autenticación de los usuarios y la gestión de permisos a las diferentes partes de la aplicación.

La tabla "clientes" almacena datos de los clientes del taller, incluyendo nombre, teléfono, correo electrónico, dirección y fecha de alta. Cada cliente puede tener uno o varios vehículos registrados.

La entidad se relaciona con los vehículos que posee cada cliente mediante la clave foránea (id_cliente) en la tabla vehículos.

La información sobre los vehículos se normaliza en dos tablas: "marcas" y "modelos", donde cada modelo pertenece a una marca específica (id_marca). Esto permite mantener consistencia y evitar duplicidades en la información de vehículos.

Cada vehículo este asociado a un cliente y a un modelo y marca específico. También tiene una matrícula que lo diferencia de los demás vehículos. También se almacena información adicional como año de reparación. Esto permite tener un registro completo de vehículos y vincularlo directamente con las reparaciones hechas sobre él.

La tabla "tipos_reparacion" contiene los diferentes servicios que ofrece el taller, junto con las estimaciones de horas y los costes. Esta tabla actúa como referencia de las reparaciones reales hechas sobre los vehículos.

En la tabla "reparaciones" se relacionan cada vehículo con un tipo de reparación, almacena información sobre el estado del trabajo, si está pendiente, en progreso, finalizado o entregado, la fecha de inicio de la reparación, la fecha de fin, las horas reales invertidas y el importe total de la reparación.

Todas las relaciones entre tablas utilizan claves foráneas, esto garantiza que cada reparación corresponde a un vehículo existente, cada vehículo corresponde a un cliente registrado, cada modelo pertenece a una marca y cada usuario tiene un rol definido en el sistema. La base de datos está normalizada hasta tercera forma normal (3FN) por lo que no existen datos redundantes, cada atributo depende únicamente de la clave primaria de su tabla.

En cuanto a la separación de roles y usuarios, esto nos permite flexibilidad en la gestión de permisos sin duplicar información.

Para la creación de la base de datos para el taller, se ha buscado *dataset* ya creadas que incluyeran marcas de coches y modelos. Viendo las opciones disponibles en la red, se ha optado por poblar las tablas marcas y modelos a partir del *dataset* abierto "*All vehicles model*" disponible en *OpenDataSoft*. Contine información de fabricantes y modelos de vehículos a nivel global. Los datos fueron limpiados y normalizados para adaptarse al modelo relacional de *MecanicSync*. Esta limpieza se hizo mediante un script en Python y usando la librería "pandas". Para que el uso de datos de esta aplicación no sea muy pesado, se ha limitado a 50 marcas con sus correspondientes modelos. Por ello, primero se han importado los datos del archivo ".csv" a una tabla intermedia y luego se han insertado los datos normalizados a las tablas. A continuación, se ha eliminado esa tabla temporal para una mayor limpieza de la base de datos.

Para poder probar la funcionalidad de la aplicación y corregir posibles errores, se han insertado datos ficticios de clientes, y usuarios. En la tabla roles, en esta primera versión de la aplicación, solo existen administrador y mecánicos. Más adelante se podría escalar a administrativos, recepcionista, clientes y otros roles, para que todos ellos puedan acceder a la aplicación y a los datos que se consideren oportunos. Solo el administrador tendrá el poder de crear roles y acceder a todas las partes de la aplicación.

También se han insertado ejemplos de tipos de reparación y reparaciones asociadas a clientes y vehículos, permitiendo probar la funcionalidad completa de la aplicación.

- API REST (Spring Boot).

Como ya se ha comentado anteriormente se ha creado una aplicación para Android que interactúa con la base de datos mediante el desarrollo de una *API REST* usando *Spring Boot*. Esta capa centraliza la lógica de negocio de forma estructurada y segura y permite que la app consuma datos mediante *endpoints* siguiendo el estándar JSON.

En la arquitectura cliente- servidor, el cliente es la aplicación Android, el servidor la *API REST* (*Spring Boot*) y la base de datos está en *MYSQL*.

La API sigue una arquitectura basada en controladores "REST" (*@RestController*), utilizando el "patrón DTO" (*Data Transfer Object*) para transferir únicamente la información necesaria entre el servidor y el cliente, evitando exponer directamente las entidades de la base de datos.

Se han creado distintos controladores según la entidad que gestionan. Cada controlador se encarga de exponer los *edpoints* relacionados con su dominio siguiendo principios de separación de responsabilidades.

"UsuarioController" es el controlador que se utiliza para la gestión completa de usuarios. Entre las funcionalidades implementadas podemos encontrar la de mostrar un listado de usuarios u obtener un determinado usuario por Id o por correo electrónico. Esto nos ayudará a la validación del usuario y de la contraseña, la creación de nuevos usuarios, la actualización de datos y la eliminación de usuarios.

Para la autenticación se ha implementado un sistema de *login* mediante el *endpoint* "`POST /api/login`". El login valida el email y la contraseña y devuelve el estado de autenticación, el rol del usuario (Administrador o mecánico) y el nombre del usuario.

Actualmente las contraseñas se almacenan en texto plano para simplificar el desarrollo, aunque se recomienda el uso de algoritmos de hash como BCrypt en entornos de producción

El "ClienteController" permite gestionar toda la información relacionada con los clientes. Al igual que con usuario, tenemos una serie de *endpoints* que nos permitan obtener una lista de los clientes, tener acceso a un cliente por un id, crear un nuevo cliente, actualizarlo o eliminarlo.

Además de estas funciones, también se han implementado funcionalidades más avanzadas como obtener los vehículos o las reparaciones de un determinado cliente, así como los clientes que tienen vehículos en el taller en el momento de la consulta.

"VehiculoController" gestiona los vehículos asociados a los clientes. Se ha vuelto a implementar las funcionalidades completas de un ciclo CRUD, y he añadido otras funcionalidades más específicas como buscar vehículos por matricula, por id del cliente, vehículos que estén actualmente en el taller y, por último, un *endpoint* que muestre las reparaciones realizadas sobre un determinado vehículo.

Para la gestión de las reparaciones su usa "ReparacionesController" y además de hacer el ciclo completo CRUD también se pueden hacer consultas por vehículo, por el estado de la reparación o buscar todas las reparaciones realizadas sobre una matrícula.

"MarcaController" permite obtener el listado de las marcas, selecciona una por el id y a partir de las marcas mostrar los modelos de la misma. "ModeloController" muestra los modelos por el id de la marca.

"TipoReparacionController" permite obtener los tipos de reparaciones disponibles. E incluye el nombre del servicio a realizar, las horas estimadas, el precio por hora de la mano de obra y el coste estimado de los materiales.

Para que el administrador y los mecánicos tengan una visión global del taller y de lo que pasa en él, en tiempo real, se ha implementado un "DashboardController". Con el *endpoint* "`GET/api/dashboard"` podemos obtener el número total de clientes de vehículos y de reparaciones y también ver las reparaciones en función de su estado.

El uso de "DTOs" está considerado como buenas prácticas, ya que evita exponer directamente las entidades, reduce el tamaño de las respuestas y controla los datos que se envía al cliente (la aplicación Android).

Para el manejo de errores la Api utiliza códigos HTTP estándar. El 200 OK para (operaciones correctas), 201 CREATED (recurso creado), 204 NO CONTENT (sin contenido), 400 BAD REQUEST (error en datos), 401 UNAUTHORIZED (error de autenticación) y 404 NOT FOUND (recurso no encontrado).

- Aplicación Android.

La aplicación móvil de "*MecanicSync*" ha sido desarrollada para dispositivos Android, actuando como interfaz principal para los empleados del taller. Su objetivo es ofrecer una experiencia de usuario intuitiva y eficiente, permitiendo gestionar clientes, vehículos y reparaciones en tiempo real.

La aplicación sigue una arquitectura moderna basada en el patrón MVVM (Modelo-Vista-ViewModel), junto con una clara separación por capas, lo que permite mejorar su mantenimiento, la escalabilidad y el testeo del sistema.

Por ello, se han definido la capa de datos, que será la encargada de la comunicación con la *API Rest*. Esta capa encapsula el acceso a los datos por lo que en un futuro se podría modificar la fuente de datos sin afectar al resto de la aplicación.

Se utiliza "Retrofit", que usa como cliente HTTP para consumir la API, *Repository* que actúa de intermediario entre la API y la capa del dominio o *domain* y *Client y Response* para gestionar las llamadas y respuestas de red. En la capa "Core.di*"* se encuentra el módulo de *Retrofit* o "RetrofitModule*"*, que se encarga de proporcionar la configuración del cliente HTTP y los servicios de la API. Esto simplifica mucho la comunicación con la *API Rest*.

La capa de dominio o *domain*, contiene la lógica de negocio de la aplicación y cada caso de uso o "UseCase" representa una operación concreta, como *login*, obtener clientes, crear reparaciones, etc.

La capa de presentación o UI es la responsable de la interacción con el usuario. Está compuesta por "Model" o modelos específicos para la UI, por las pantallas de la aplicación o "Screens" y por los "ViewModels" que gestionan el estado de la UI y la comunicación con los "UseCase" y permiten mantener el estado ante cambios de configuración.

De este modo, la lógica de negocio, la fuente de datos y la interfaz de usuario, quedan separadas.

Se ha usado "Hilt" para para la inyección de dependencias, facilitando la creación y reutilización de componentes.

La visualización del CV se ha implementado mediante un "Intent" junto con "FileProvider", permitiendo abrir el documento PDF desde el almacenamiento interno de la aplicación en un visor externo compatible. Este enfoque garantiza la seguridad en el acceso a archivos internos, ya que evita la exposición directa de rutas del sistema de archivos y permite aprovechar las aplicaciones instaladas en el dispositivo, mejorando la compatibilidad, el rendimiento y simplificando la implementación. Adicionalmente, se ha contemplado la posibilidad de visualización mediante "WebView" en caso de utilizar una versión remota del documento alojada en la nube.

4.2. Documentación de usuario final.

A continuación, pasamos a detallar las diferentes pantallas de la aplicación y la funcionalidad de cada una de ellas, para facilitar al usuario final su comprensión.

Para la autenticación y el control de acceso a la aplicación, tenemos una pantalla inicial de *login*, donde el usuario tendrá que introducir el email de usuario (1) y la contraseña (2). En función del rol que desempeñe el usuario, administrador o mecánico, tendrán acceso a unas partes de la aplicación o a otras.

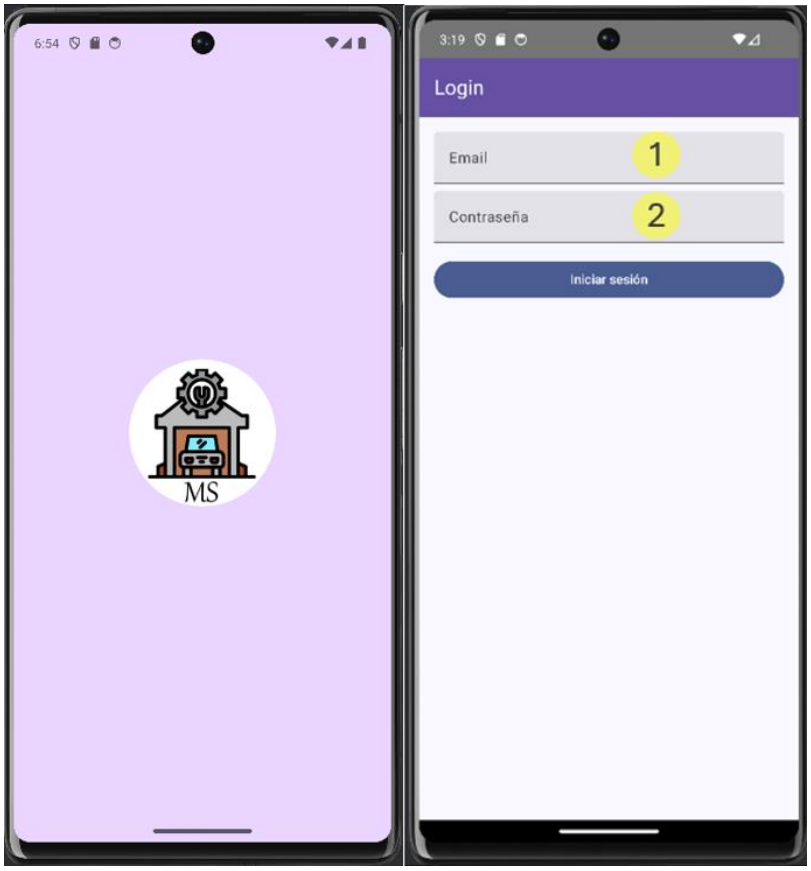


Figura 10: Ventana Login e inicio de sesión.

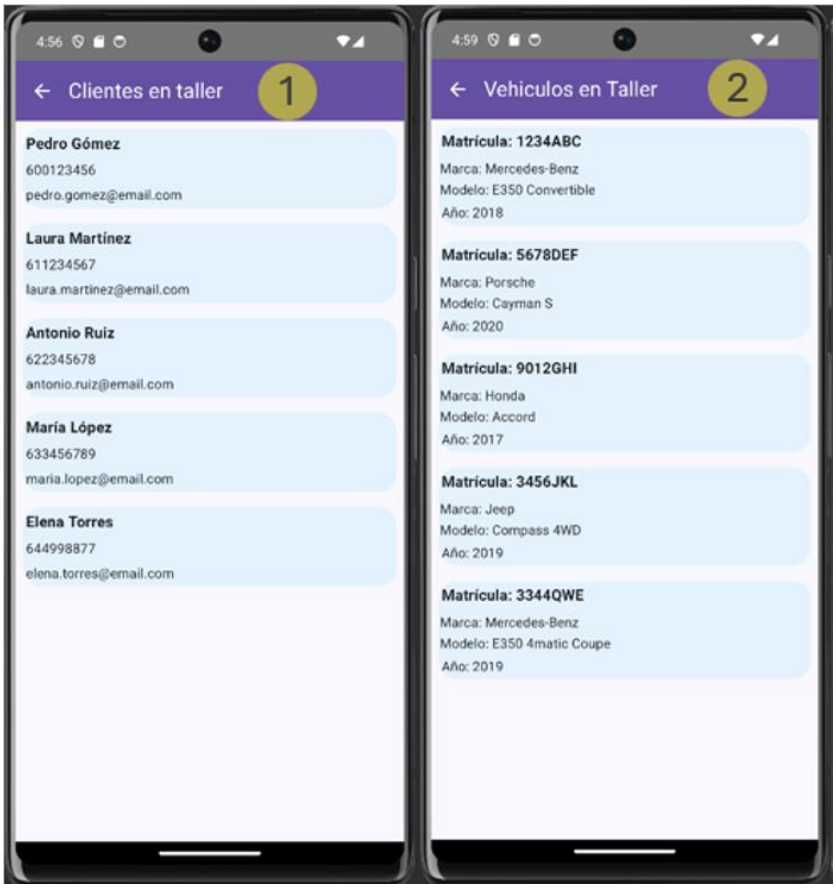
El ***administrador***, tendrá un acceso completo a los módulos de la aplicación. Una vez introducidos los datos de acceso, el administrador, podrá acceder a la pantalla principal o Dashboard. Esta pantalla actúa como panel de control del taller y se divide en dos secciones.

Figura 11: Dashboard Administrador

La primera de ellas es "En taller" (1). En esta sección se puede ver el número de clientes que tienen un vehículo en el taller, el número de vehículos en proceso de reparación y el estado de las reparaciones en taller (pendiente, en proceso y finalizadas). Estos datos se obtienen mediante el api y se actualizan dinámicamente, proporcionando una visión global del taller.

La segunda sección es la de "Accesos rápidos" (2), que permite navegar entre los distintos módulos de la aplicación, usuarios, clientes, vehículos, reparaciones e histórico de reparaciones.

En la primera sección, al seleccionar una de las "Card "podemos consultar los datos a los que se refiere.



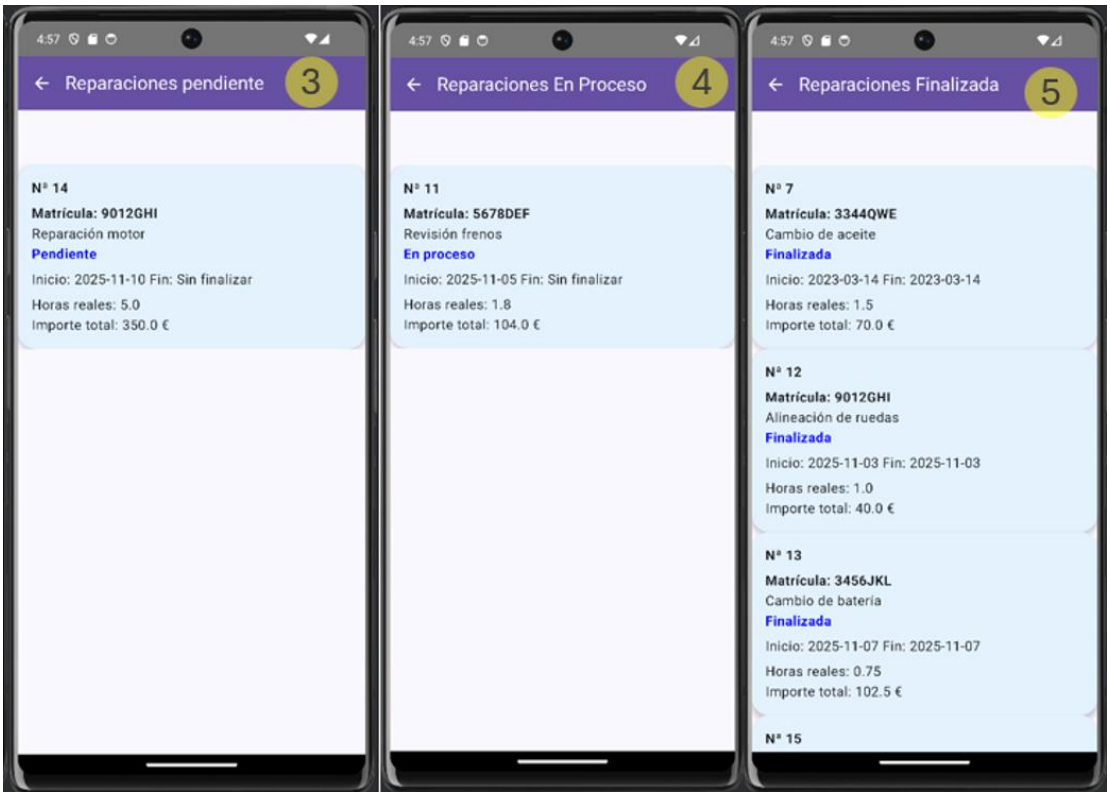


Figura 12: Datos de las Card de "En taller"

En el módulo de "usuarios", el administrador, podrá consultar (1) y editar (2) nombre y apellidos, correo y rol de todos los usuarios de la aplicación. Además, podrá añadir (3) nuevos usuarios y/o eliminarlos (4).

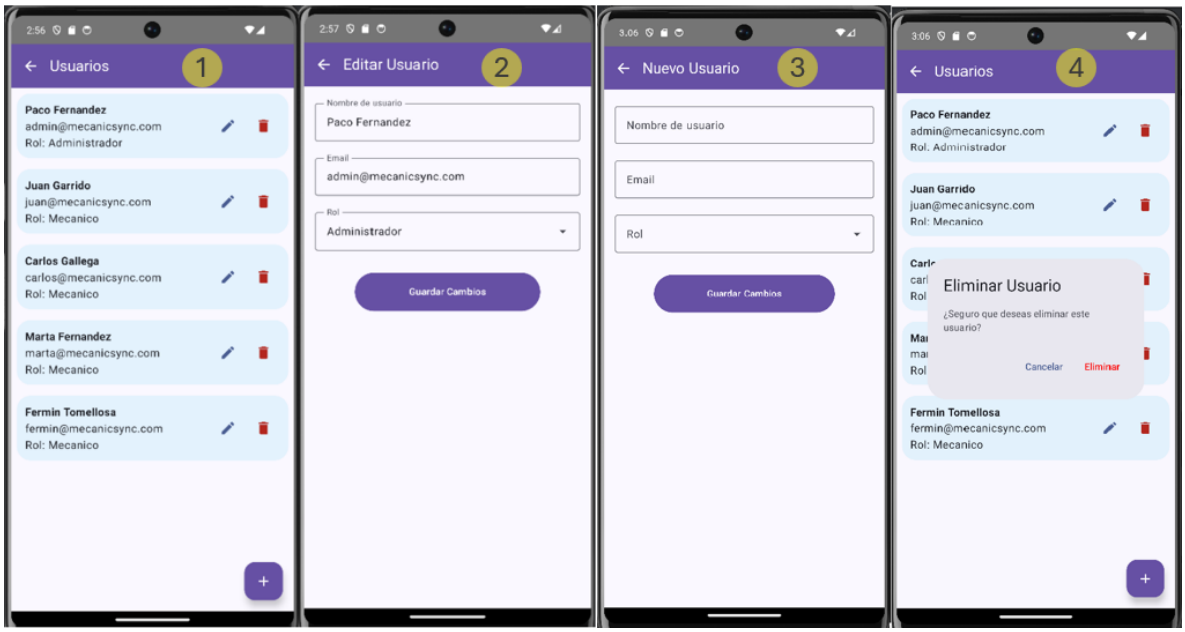


Figura 13: Flujo CRUD de Usuario

En el módulo de "clientes", aparecerán todos los clientes (1) dados de alta en el taller, y al igual que en los demás módulos se podrán editar (2), eliminar (4) y crear usuarios (3).

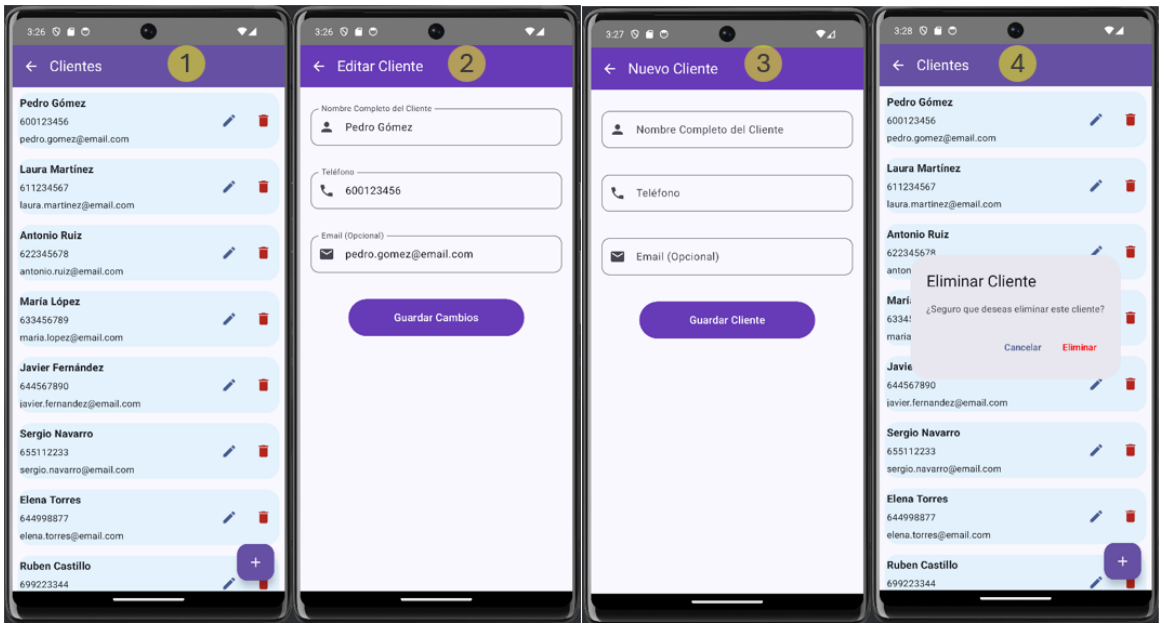


Figura 14: Flujo CRUD de Cliente

En el módulo "vehículo", se podrá consultar (1) todos los vehículos dados de alta en el taller, con las matrículas, marca y modelo, además de otros datos. Se podrán editar (2), para por ejemplo cambiar el dueño del vehículo. Se podrán dar de alta nuevos vehículos (3), referenciándolos a una matrícula y se podrán eliminar vehículos (4).

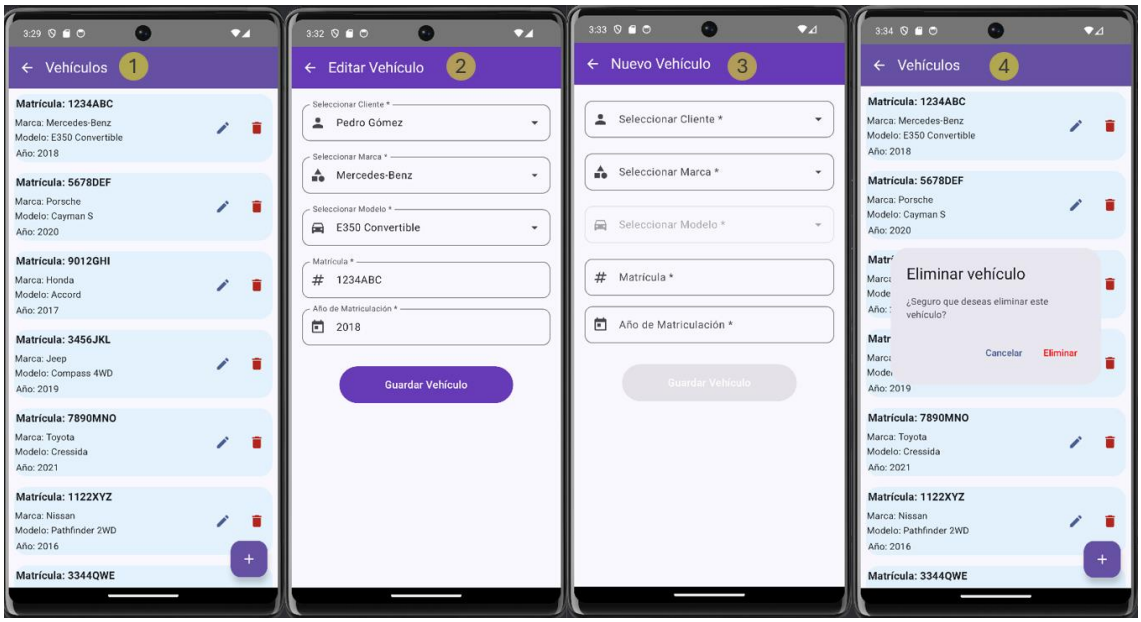


Figura 15: Flujo CRUD Vehículo

En la pantalla de “reparaciones” aparecerán todas las reparaciones (1) realizadas en el taller, ordenadas por el estado de la misma. Es decir, aparecerán más arriba las que estén aún “pendientes”, luego las que están “en proceso”, a continuación, las “finalizadas” y por último las “entregadas”. En editar reparaciones (2) solo se podrá modificar el estado de la reparación del vehículo (3), la descripción adicional, por si el mecánico o el administrador quieren añadir algún comentaría sobre esta reparación especifica, las horas reales o el importe. Una vez que el mecánico o administrador cambie el estado a “finalizado” (3), se habilitará la pestaña de fecha de fin (4). Esto busca agilizar el trabajo del mecánico en el taller.

También se podrán dar de alta nuevas reparaciones. En “Seleccionar vehículo” (6) se abrirá un desplegable para seleccionar las matrículas que ya se habrán dado de alta anteriormente y cargarán dos campos más (7), la marca y el modelo asociado a esa matrícula. “Tipo de Reparación” (8), también mostrará un desplegable con los tipos de reparación que tenemos en nuestra base de datos. Por último, se podrá borra una determinada reparación.

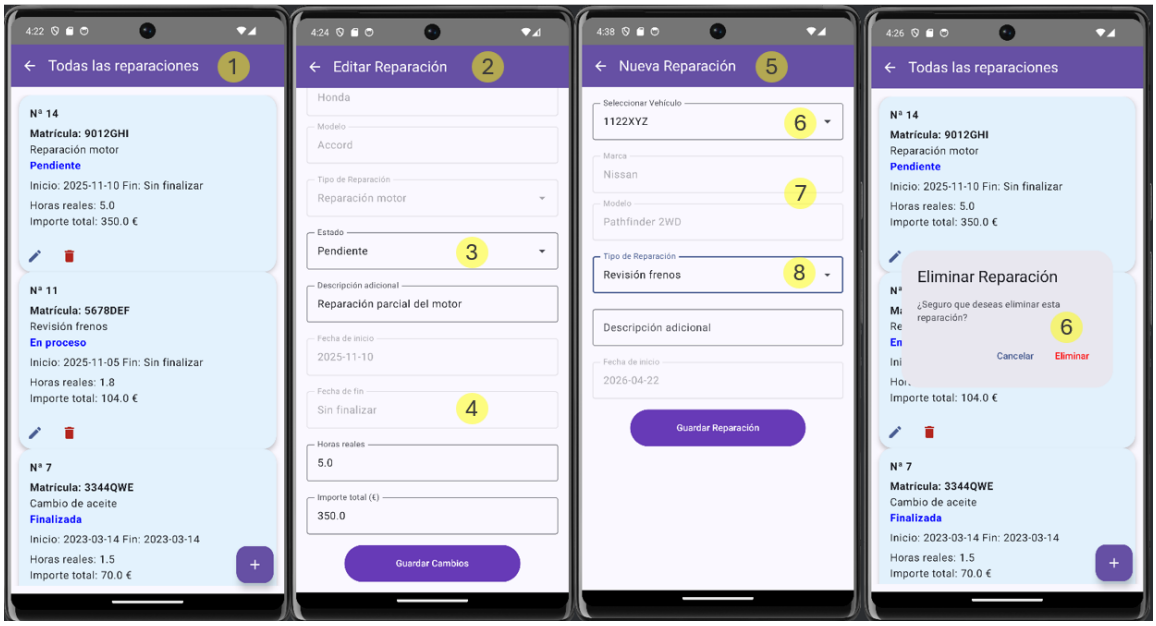


Figura 16: Flujo CRUD Reparaciones

En histórico de reparación (1), podemos hacer una búsqueda por la matrícula del vehículo. De esta manera aparecerá el nombre del cliente, la matrícula y marca del vehículo (3) y una lista de las reparaciones hechas sobre el vehículo. También aparece el importe de la reparación y el estado de las mismas, de forma muy visual, con una escala de colores (5).

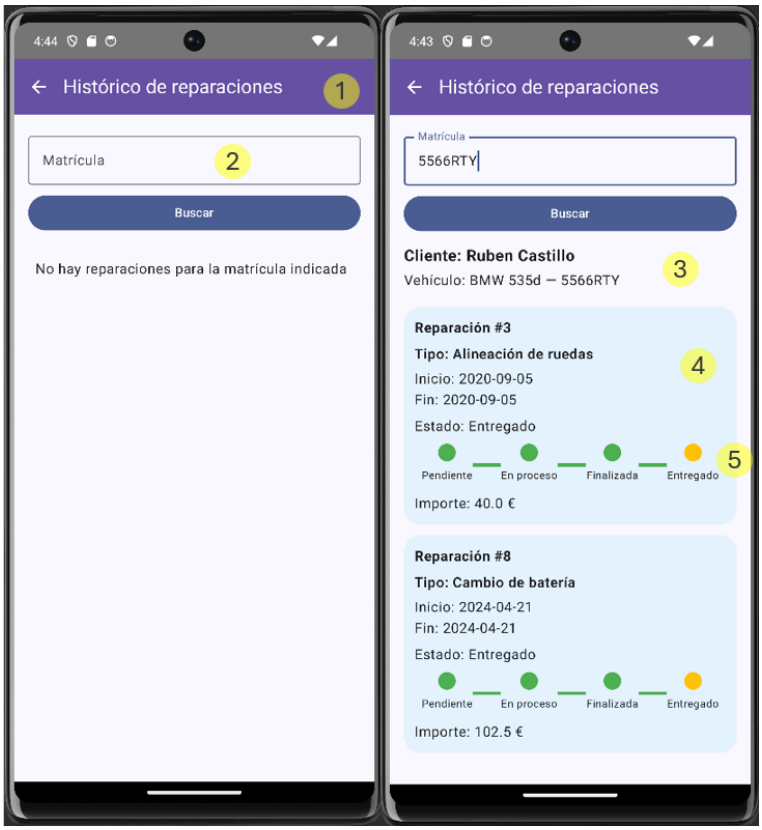


Figura 17: Pantalla de Histórico de Reparaciones

Por último, existe un desplegable en la barra superior de opciones, en el que administrador y mecánico podrán consultar su perfil (1) y cerrar la sesión (2). En el caso del administrador también podrá consultar los datos "Sobre mi" (3).

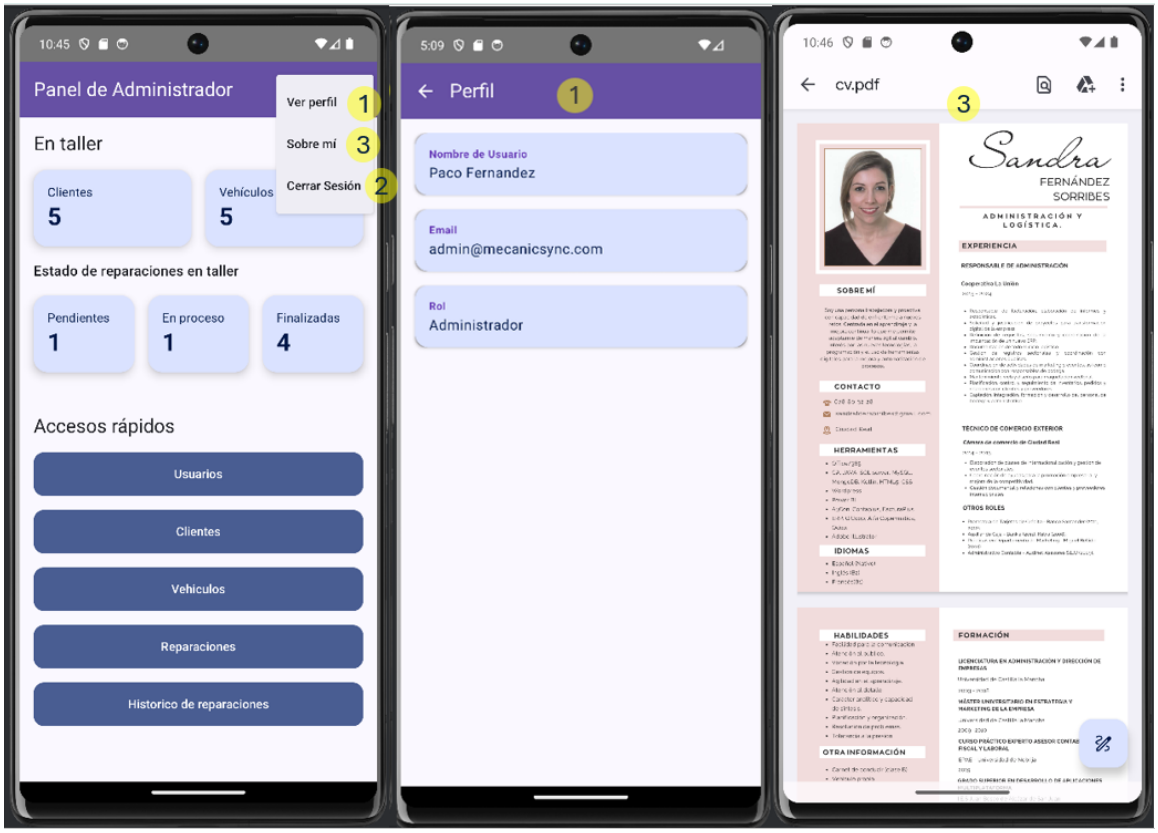


Figura 18: Opciones de barra de Menú

Si quien inicia sesión tiene el rol de mecánico, no tendrá acceso a todas las pantallas de la aplicación. Solo tendrá una sección, que se llamará tareas (1) y que contendrá dos módulos, el de reparaciones (2) y el de histórico de reparaciones (3), ya que se presupone que serán los únicos necesarios para desarrollar su trabajo. Dar de alta clientes o vehículos será responsabilidad de la administración de la empresa. También tendrán el desplegable en la barra superior de opciones (4) para consultar sus datos de perfil y cerrar sesión.

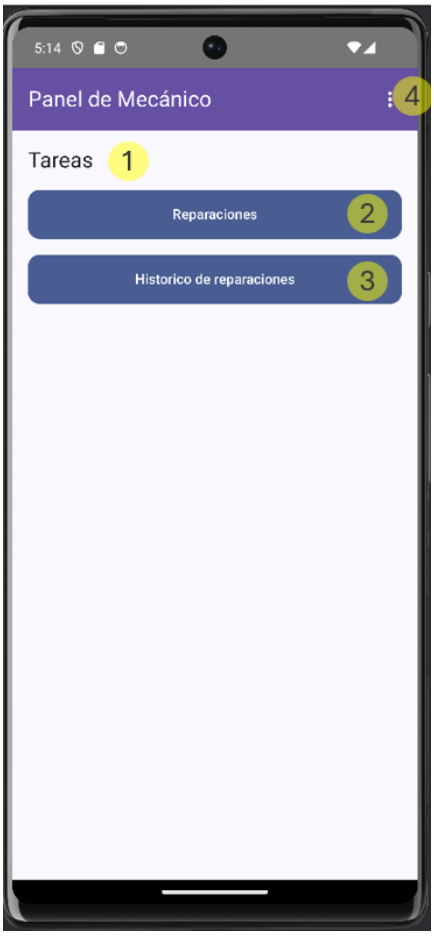


Figura 19: Dashboard de Mecánico.

La funcionalidad de estas dos tareas será completa, al igual que el del administrador, ya que será necesario para poder gestionar su trabajo.

4.3. Manual de instalación y configuración.

En este apartado se describen los pasos necesarios para la instalación, configuración y puesta en funcionamiento del sistema *MecanicSync*. Durante la fase de desarrollo en la que se encuentra actualmente la aplicación, esta se ejecutará en un entorno local. Este entorno permite realizar pruebas, depuración y validación del sistema completo.

El sistema está compuesto por tres partes principales:

- Base de datos MYSQL
- API REST desarrollada con *Spring Boot*
- Aplicación Android.

En un entorno de producción, la arquitectura del sistema cambiaría para adaptarse a un uso real por parte de los usuarios finales. La aplicación se distribuiría como un paquete instalable ".APK" a través de plataformas de gestión como "Google Play Store". La *API Rest* se debería alojar en un servidor accesible desde internet, al igual que la base de datos. Los requisitos del sistema serían otros.

En la actualidad, en la fase de desarrollo, en la que se encuentra la aplicación, debemos tener en cuenta los requisitos del sistema necesarios para su instalación.

- Requisitos del Sistema:

Para la correcta ejecución del sistema se requieren los siguientes elementos de software:

- MySQL Server 8.0 o superior
- MySQL Workbench (opcional, para gestión visual)
- Java JDK 17 o superior
- Maven
- Android Studio
- Dispositivo Android o emulador

- Instalación de la base de datos:
  - Abrir MySQL Workbench o consola MySQL.
  - Ejecutar el script SQL proporcionado en el proyecto (*scriptfernandez.sql*). Este script crea la base de datos *basefernandez*, las tablas del proyecto, define las claves primarias y foráneas, inserta datos iniciales y carga datos de un dataset para marcas y modelos de coches.
  - Verificar que las tablas se han creado correctamente.

- Configurar API REST
  - Abrir el proyecto de *Spring Boot* en el entorno de desarrollo (por ejemplo, IntelliJ o Eclipse).
  - Configurar el archivo application.properties con los datos de conexión a la base de datos:

```
spring.datasource.url=jdbc:mysql://localhost:3306/basefernandez?useSSL=false&serverTimezone=UTC
spring.datasource.username=usuario
spring.datasource.password=contraseña
spring.jpa.hibernate.ddl-auto=none
spring.jpa.show-sql=true
```

  - Ejecutar la aplicación: La Api estará disponible en: http://localhost:8080/api

- Instalación de la aplicación Android
  - Abrir el proyecto en Android Studio.
  - Esperar a que "Gradle" sincronice las dependencias automáticamente.
  - Configurar la URL base de la API en "Retrofit" (si es necesario):

```
baseUrl = http://10.0.2.2:8080/
```

Nota: *10.0.2.2 se utiliza cuando se ejecuta en emulador. Para dispositivos físicos usar la IP del servidor.*

  - Ejecutar la aplicación en un emulador Android o en un dispositivo físico conectado, según el punto anterior.

- Credenciales de acceso:

Se incluyen usuarios de prueba para acceder a la aplicación:

Administrador

- Email: admin@mecanicsync.com
- Contraseña: admin123.

Mecánico

- Email: juan@mecanicsync.com
- Contraseña: juan123

4.4. Consideraciones adicionales.

- Es necesario que la API esté en ejecución antes de iniciar la aplicación Android.
- El dispositivo Android debe tener conexión a la misma red que el servidor.
- En caso de error de conexión, verificar:
  - URL de la API
  - Estado del servidor
  - Configuración de red

5. MANTENIMIENTO Y EVOLUCIÓN.

5.1. Identificación de posibles mejoras y evolución del proyecto.

Aunque el sistema desarrollado es funcional, existen diversas líneas de mejora que permitirían ampliar sus capacidades:

- Desarrollar una aplicación a la que los clientes también puedan acceder con su propio usuario y puedan visualizar sus reparaciones.
- Implementación de un sistema de gestión de citas. Solicitud de citas directamente por el cliente y sistema de recordatorio de las mismas.
- Conexión con un sistema de facturación.
- Envío automático de informes y facturas a clientes.
- Mejora de la seguridad mediante el uso de contraseñas encriptadas.
- Implementación de autenticación mediante tokens (JWT).

Estas mejoras permitirían evolucionar *MecanicSync* hacia una solución más completa y adaptada a entornos reales de producción.

A pesar de que la aplicación cumple con los objetivos planteados, existen ciertas limitaciones:

- El sistema de autenticación no implementa cifrado de contraseñas (pendiente uso de BCrypt).

- No se ha implementado control de concurrencia en la base de datos.
- La aplicación no incluye notificaciones en tiempo real.

5.2. Mejoras futuras.

Como líneas de mejora se plantean:

- Implementación de caché local
- Uso de paginación en listados grandes
- Optimización de consultas complejas
- Implementación de notificaciones en tiempo real
- Desarrollar nuevos roles de usuario

6. CONCLUSIONES.

6.1. Evaluación del proyecto.

El desarrollo de *MecanicSync* ha permitido diseñar e implementar una solución completa para la gestión de talleres mecánicos, abarcando desde la persistencia de datos hasta la interfaz de usuario final.

El sistema desarrollado cumple con el objetivo de digitalizar la gestión del taller, permitiendo registrar clientes, vehículos y reparaciones, así como consultar el estado del trabajo en tiempo real. La integración entre la base de datos, la *API REST* y la aplicación Android ha permitido construir una solución funcional, estructurada y escalable.

Desde un punto de vista técnico, el proyecto destaca por la correcta aplicación de una arquitectura cliente-servidor, así como por el uso de tecnologías actuales como *Spring Boot* en el *backend y Jetpack Compose* en el desarrollo móvil.

6.2. Cumplimiento de objetivos y requisitos.

Los objetivos planteados al inicio del proyecto han sido alcanzados de forma satisfactoria:

- Se ha conseguido digitalizar la gestión de reparaciones del taller.
- Se ha implementado un sistema completo de gestión de clientes y vehículos.
- Se ha desarrollado un sistema de control del estado de las reparaciones.
- Se ha implementado un sistema de autenticación con roles diferenciados.
- Se ha desarrollado una interfaz intuitiva y funcional en Android.
- Se ha incorporado un módulo adicional "Sobre mí" para la presentación profesional.

Además, los requerimientos no funcionales también se han cumplido, destacando la usabilidad de la aplicación, la escalabilidad del sistema y la correcta separación de capas en la arquitectura.

6.3. Lecciones aprendidas y recomendaciones para futuros proyectos.

Durante el desarrollo del proyecto se han adquirido conocimientos clave tanto a nivel técnico como metodológico.

A nivel técnico, se ha profundizado en:

- El diseño y normalización de bases de datos relacionales.
- El desarrollo de *APIs REST* con *Spring Boot*.
- La arquitectura de aplicaciones móviles basada en "MVVM".
- El uso de herramientas modernas como "Retrofit" y "Hilt".
- La importancia del uso de "DTOs" para la transferencia de datos.

A nivel práctico, se ha comprendido la importancia de:

- Diseñar correctamente la arquitectura antes de implementar.
- Separar responsabilidades entre capas.
- Gestionar errores y validar datos.
- Mantener un código limpio y estructurado.

Uno de los principales retos ha sido la integración entre las diferentes capas del sistema, especialmente en la comunicación entre la aplicación Android y la *API REST*.

También se han presentado dificultades en la gestión de estados en la interfaz de usuario, así como en la correcta estructuración de la aplicación siguiendo el "patrón MVVM".

Otro aspecto relevante ha sido la toma de decisiones técnicas, como la elección del método para visualizar el currículum vitae, donde finalmente se optó por utilizar un *Intent* del sistema para mejorar la compatibilidad y simplificar la implementación.

Este proyecto ha supuesto una experiencia muy enriquecedora, permitiendo aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo.

Se ha conseguido desarrollar una aplicación real, con una estructura profesional y orientada a un caso de uso concreto, lo que aporta un valor significativo de cara al ámbito laboral.

Además, el proyecto ha servido para reforzar habilidades como la organización, la resolución de problemas y la toma de decisiones técnicas.

7. BIBLIOGRAFÍA/WEBGRAFÍA Y REFERENCIAS.

Para el desarrollo del proyecto *MecanicSync* se han utilizado diversas fuentes técnicas y documentación oficial, fundamentales para el diseño, implementación y validación de la aplicación.

7.1. Fuentes utilizadas en el proyecto.

- *Spring Boot* – Documentación oficial
  https://spring.io/projects/spring-boot
- Android Developers – Desarrollo de aplicaciones Android
  https://developer.android.com
- *MySQL* – Documentación oficial
  https://dev.mysql.com/doc/
- *MySQL Workbench* – Modelado y diseño de bases de datos
  https://dev.mysql.com/downloads/workbench/
- *Retrofit* – Cliente HTTP para Android
  https://square.github.io/retrofit/
- *Hilt* – Inyección de dependencias
  https://developer.android.com/training/dependency-injection/hilt-android
- *Jetpack Compose* – Desarrollo de interfaces modernas
  https://developer.android.com/jetpack/compose
- Documentación sobre arquitectura MVVM:
  https://developer.android.com/topic/architecture
- *Guías REST API*: https://restfulapi.net
- Software de gestión para talleres mecánicos:
  https://www.programatallermecani.com/
- https://autosofttaller.com/index.html
- https://gdtaller.com/

7.2. Herramientas utilizadas.

- *MySQL Workbench*
- *IntelliJ IDEA*
- *Android Studio*
- *Postman*
- *GitHub*

Índice de figuras

Figura 1: Diagrama entidad-relación 7
Figura 2: Prueba de consulta de todos los vehículos usando Postman 8
Figura 3: Prueba de inserción de nuevo vehículo en la base de datos usando *Postman* 9
Figura 4: Resultados de la base de datos posteriores a la inserción de un nuevo vehículo usando *Postman*. 9
Figura 5: Prueba de consulta por estado de una reparación usando *Postman* 10
Figura 6: Prueba de creación de una reparación sobre un vehículo inexistente. Respuesta 400 11
Figura 7: Prueba de creación de una reparación con tipo de reparación no válido. Respuesta 400 12
Figura 8: Prueba de actualización de un recurso inexistente. Respuesta 404 12
Figura 9: Prueba de eliminación de un recurso inexistente. Respuesta 404. 13
Figura 10: Ventana Login e inicio de sesión. 20
Figura 11: Dashboard Administrador 21
Figura 12: Datos de las Card de "En taller" 22
Figura 13: Flujo CRUD de Usuario 23
Figura 14: Flujo CRUD de Cliente 23
Figura 15: Flujo CRUD Vehículo 24
Figura 16: Flujo CRUD Reparaciones 25
Figura 17: Pantalla de Histórico de Reparaciones 25
Figura 18: Opciones de barra de Menú 26
Figura 19: Dashboard de Mecánico 27